Université Tahri Mohamed de Béchar
Faculté des sciences exactes
Département des Mathématiques et Informatique
2ême Année Ingénieur Informatique

Matière:

Algorithmiques et complexité

Cours n° 03

Enseignant : Benyamina Ahmed

Matière: Algorithmiques et complexité

Programme de la matière :

I- Rappels

II- Notion de complexité algorithmique

III- Algorithmes de tri

1. **Présentation**
2. **Tri à Bulles (Bubble-Sort)**
3. Tri par sélection (Selection-sort)
4. Tri par insertion (Insertion-sort)
5. Tri fusion (Merge-sort)
6. Tri rapide (quick-sort)
7. Conclusion

IV- Les arbres

V- Les graphes

Algorithmes de tri

1- Présentation

Trier les données:

C'est : Créer un ordre dans les données

- Ex. Ordre croissant, décroissant des valeurs numériques
- Ordre croissant des mots, …

Exemple du tri de données:

On a la suite de nombre entiers suivante :

5 , 8 , -3 , 6 , 42 , 2 , 101 , -8 , 42 , 6

et l'on souhaite les classer par ordre croissant.

La suite précédente devient alors après le tri (classement) :

-8 , -3 , 2 , 5 , 6 , 6 , 8 , 42 , 42 , 101

Il s'agit en fait d'une nouvelle suite obtenue par une permutation des éléments de la première liste de telle façon que les éléments résultants soient classés par ordre : "≤"

$$-8 \leq -3 \leq 2 \leq 5 \leq 6 \leq 6 \leq 8 \leq 42 \leq 42 \leq 101$$

Algorithmes de tri

1- Présentation

Définition du Tri:

Le tri est une opération de **classement d'éléments** d'une liste selon un **ordre total défini**.

Pourquoi trier?

- Faciliter la recherche
- Faciliter la gestion en générale

Utilisation du tri

Cette opération se trouve très souvent en informatique dans beaucoup de structures de données.

Par exemple:

- Le classement de certains élèves,
- Mettre en ordre un dictionnaire,
- Trier l'index d'un livre, etc…

Deux opérations élémentaires :

1. La comparaison de deux éléments de la liste a_i et a_k, **(si $a_i > a_k$, si $a_i < a_k$, ,…)**
2. L'échange des places de deux éléments de la liste a_i et a_k **place (a_i) ⇔ place (a_k)**

Ces deux opérations seront utilisées afin de fournir une mesure de comparaison des tris entre eux.

Algorithmes de tri

2- Tri à Bulles (Bubble sort) - Présentation

Voici la liste des nombres initialement placés dans un tableau:

- L'algorithme compare les éléments adjacents A[i] et A[i+1], i = 0,1,.. du tableau un par un. Si deux éléments n'ont pas la place convenable, ils s'échangent. En conséquence, les éléments ayant des valeurs plus élevées sont déplacés vers la fin du tableau. Le pointeur ⬇ indique l'élément par rapport au suivant. Le pointeur ⬇ indique le bord droit d'un sous-tableau considéré.

Algorithmes de tri

2- Tri à Bulles (Bubble sort) - Exemple

- Comparaisons entre A[0] et A[1].
- Ces éléments ne sont pas en ordre : 8 > 4. ⇒ échange entre les éléments.

Comparaisons entre A[1] et A[2].

Ces éléments ne sont pas en ordre : 8 > 5. ⇒ échange entre les éléments.

Comparaisons entre A[2] et A[3].

Ces éléments ne sont pas en ordre : 8 > 6. ⇒ échange entre les éléments.

Algorithmes de tri

2- Tri à Bulles (Bubble sort) - Exemple

- Comparaisons entre A[3] et A[4].
- Ces éléments ne sont pas en ordre : 8 > 0. ⇒ échange entre les éléments.

Comparaisons entre A[4] et A[5].

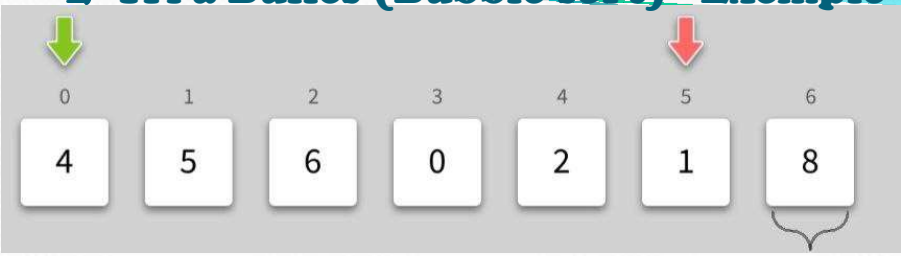
Ces éléments ne sont pas en ordre : 8 > 2. ⇒ échange entre les éléments.

Comparaisons entre A[5] et A[6].

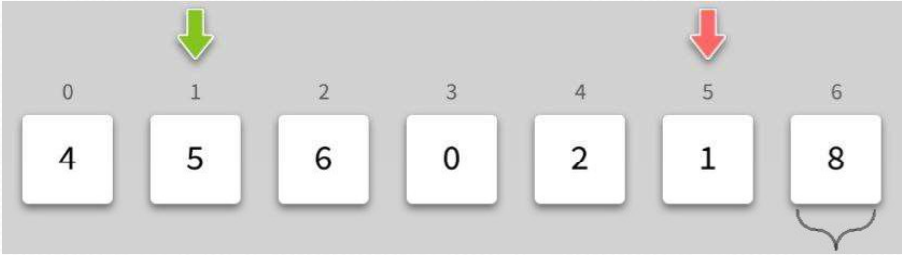
Ces éléments ne sont pas en ordre : 8 > 1. ⇒ échange entre les éléments.

Algorithmes de tri

2- Tri à Bulles (Bubble sort) - Exemple



- Comparaisons entre A[0] et A[1]. **Trié**
- Ces éléments sont en ordre : 4 < 5. ⇒ Aucune échange entre les éléments.



Comparaisons entre A[1] et A[2]. **Trié**

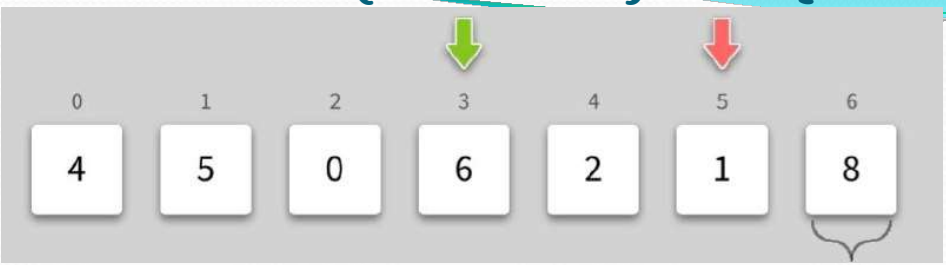
Ces éléments sont en ordre : 5 < 6. ⇒ Aucune échange entre les éléments.

Comparaisons entre A[2] et A[3]. **Trié**

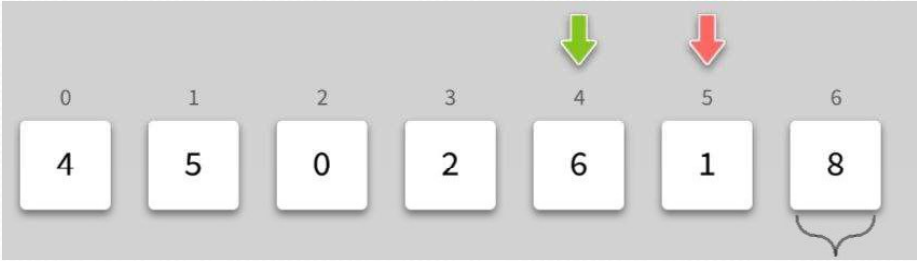
Ces éléments ne sont pas en ordre : 6 > 0. ⇒ échange entre les éléments.

Algorithmes de tri

2- Tri à Bulles (Bubble sort) - Exemple

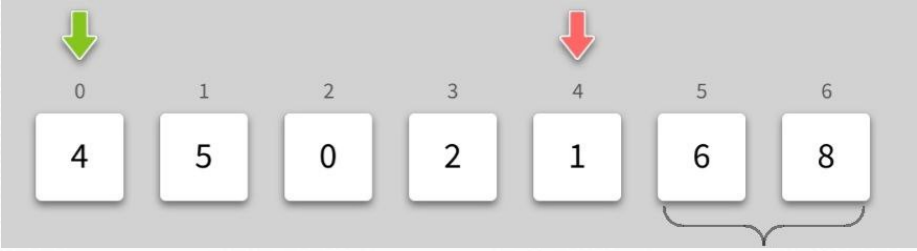


- Comparaisons entre A[3] et A[4]. **Trié**
- Ces éléments ne sont pas en ordre : 6 > 2. ⇒ échange entre les éléments.



Comparaisons entre A[4] et A[5]. **Trié**

Ces éléments ne sont pas en ordre : 6 > 1. ⇒ échange entre les éléments.

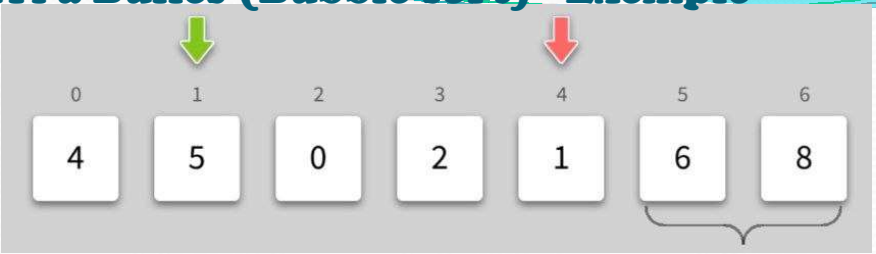


Comparaisons entre A[0] et A[1]. **Triés**

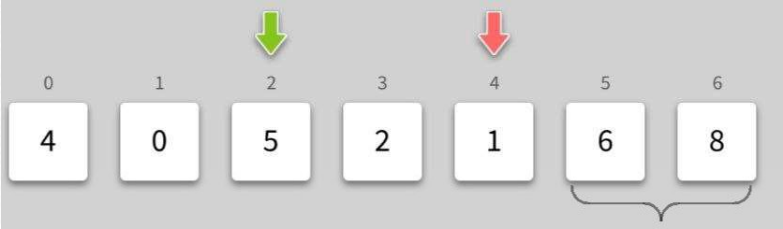
Ces éléments sont en ordre : 4 < 5. ⇒ Aucune échange entre les éléments.

Algorithmes de tri

2- Tri à Bulles (Bubble sort) - Exemple

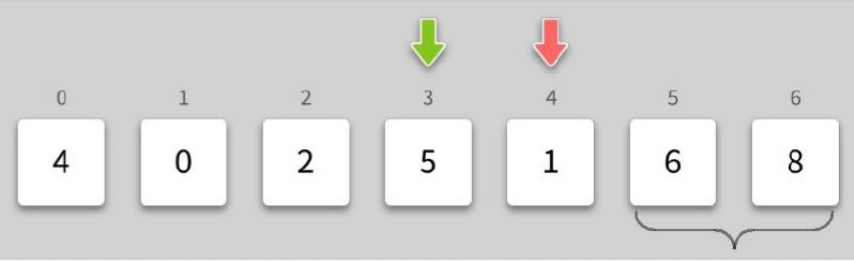


- Comparaisons entre A[1] et A[2]. **Triés**
- Ces éléments ne sont pas en ordre : 5 > 0. ⇒ échange entre les éléments.



Comparaisons entre A[2] et A[3]. **Triés**

Ces éléments ne sont pas en ordre : 5 > 2. ⇒ échange entre les éléments.

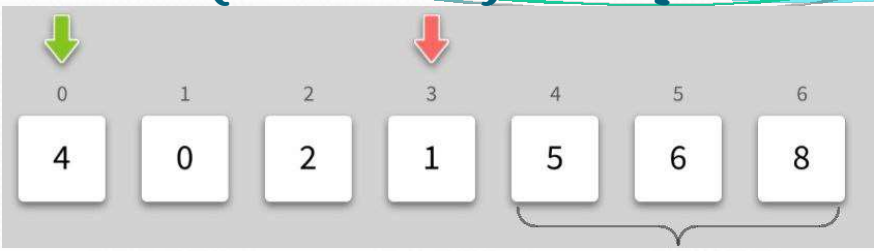


Comparaisons entre A[3] et A[4]. **Triés**

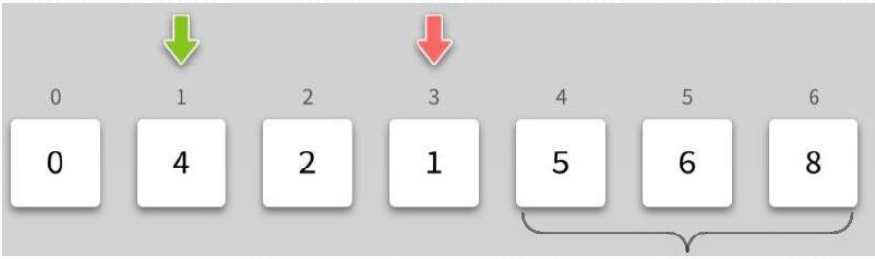
Ces éléments ne sont pas en ordre : 5 > 1. ⇒ échange entre les éléments.

Algorithmes de tri

2- Tri à Bulles (Bubble sort) - Exemple

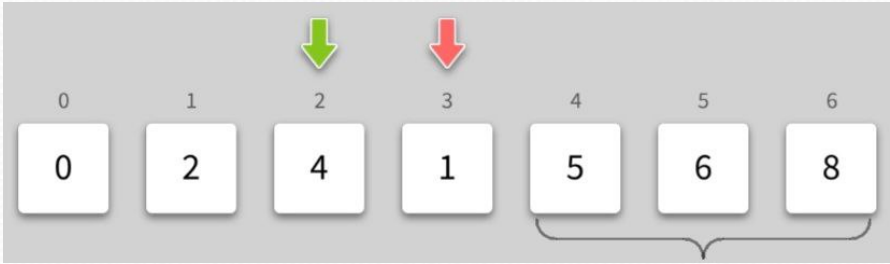


- Comparaisons entre A[0] et A[1]. **Triés**
- Ces éléments ne sont pas en ordre : 4 > 0. ⇒ échange entre les éléments.



Comparaisons entre A[1] et A[2]. **Triés**

Ces éléments ne sont pas en ordre : 4 > 2. ⇒ échange entre les éléments.

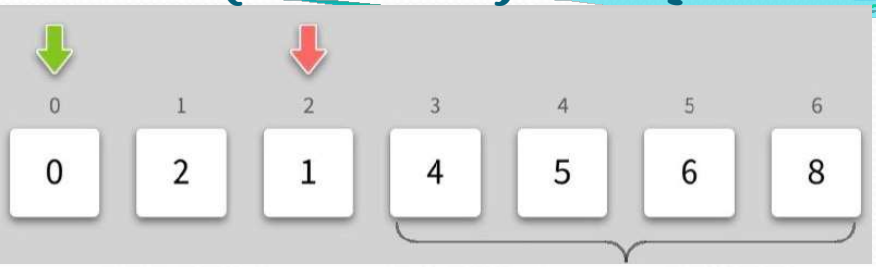


Comparaisons entre A[2] et A[3]. **Triés**

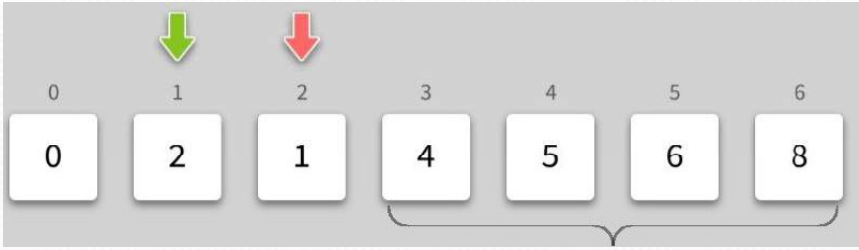
Ces éléments ne sont pas en ordre : 4 > 1. ⇒ échange entre les éléments.

Algorithmes de tri

2- Tri à Bulles (Bubble sort) - Exemple

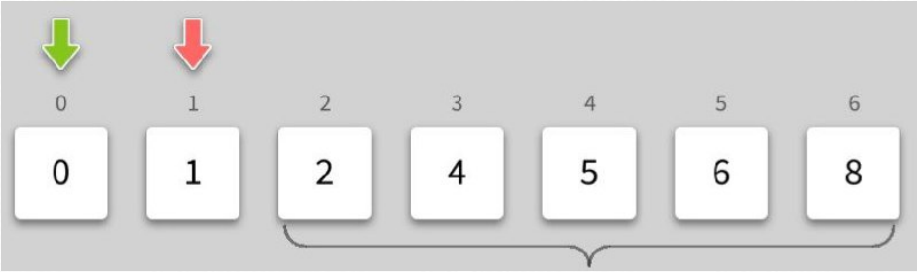


- Comparaisons entre A[0] et A[1]. **Triés**
- Ces éléments sont en ordre : 0 < 2. ⇒ Aucune échange entre les éléments.



Comparaisons entre A[1] et A[2]. **Triés**

Ces éléments ne sont pas en ordre : 2 > 1. ⇒ échange entre les éléments.

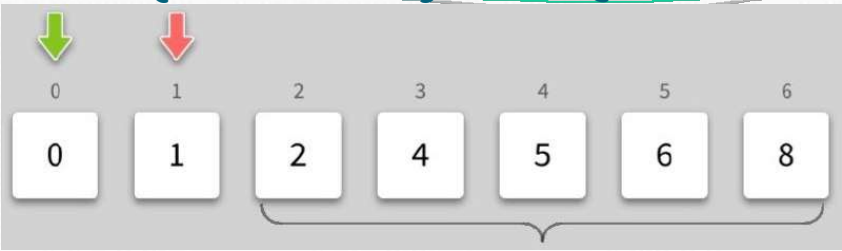


Comparaisons entre A[2] et A[3]. **Triés**

Ces éléments sont en ordre : 2 < 4. ⇒ Aucune échange entre les éléments.

Algorithmes de tri

2- Tri à Bulles (Bubble sort) - Exemple



- Comparaisons entre A[0] et A[1]. **Triés**
- Ces éléments sont en ordre : 0 < 1. ⇒ Aucune échange entre les éléments.

Donc le tableau devient trié.

Algorithmes de tri

2- Tri à Bulles (Bubble sort) - Exemple

- Voici la vidéo suivante présentant les étapes du tri à Bulle citées précédemment:

Algorithmes de tri

2- Tri à Bulles (Bubble sort) - Implémentation

La mise en œuvre du tri à bulles en Python

```
def bubbleSort(array):
    n = len(array)
      for i in range(n-1):
          for j in range(0, n-i-1):
              if array[j] > array[j+1]:
                  array[j], array[j+1] = array[j+1], array[j]
```

Le lien entre le code et la présentation:

i → ⬇

j → ⬇

Le pointeur rouge reflète les changements de la variable i depuis la boucle externe

Le pointeur vert reflète les changements de la variable j depuis la boucle interne

Algorithmes de tri

2- Tri à Bulles (Bubble sort) - Complexité

Règle: La complexité de tout algorithme de tri dépend du nombre de comparaisons.

Dans le tri à bulles, nous avons :

- Lors du premier passage on a **n-1** comparaisons effectuées : c'est à dire **n-1** comparaisons pour placer l'élément le plus haut dans sa position correcte.
- Puis dans le deuxième passage, il y a **n-2** comparaisons et le deuxième élément le plus haut est placé à sa position.
- Et ainsi de suite : n-3, n-4,….1
- Par conséquent, pour calculer la complexité du tri à bulles, nous devons calculer le nombre total de comparaisons.

Il peut être donné comme suit :

$f(n) = (n-1) + (n-2) + (n-3) + ... + 3 + 2 + 1$

$f(n) = n(n-1)/2$

$f(n) = n^2/2 + O(n/2) = O(n^2)$

Par conséquent, la complexité de l'algorithme de tri à bulles est $O(n^2)$.

Cela signifie que le temps nécessaire pour exécuter le tri à bulles est proportionnel à n^2, où n est le nombre total d'éléments dans le tableau.

-Pire des cas, Moyenne des cas et meilleur des cas: $O(n^2)$